

Coupling energy efficiency and quality for consolidation of cloud workloads

Alessandro Carrega^{a,*}, Matteo Repetto^b

^a National Inter-University Consortium of Telecommunications (CNIT) National Laboratory of Smart and Secure Networks (S2N), Italy

^b National Research Council of Italy (CNR) Institute for Applied Mathematics and Information Technologies (IMATI), Genoa, Italy

ARTICLE INFO

Keywords:

Energy efficiency
Quality of service
Workload consolidation
Infrastructure-as-a-service cloud model
Elastic cloud applications

ABSTRACT

Efficient usage of IT equipment in Data Centers requires modulating their power-consumption according to the actual workload. The most effective strategy to this aim consists in consolidating as many applications as possible on the smallest number of servers, so that idle devices can be shut down or put in low-power states. Usually, this process is driven by computing and networking resources requested by each application (e.g., CPU and RAM) and applies a certain degree of overcommitment, assuming that such resources are not fully used continuously. However, this approach is critical with real workload patterns, which usually change over time; as a matter of fact, consolidation in real scenarios often leads to either low efficiency or violations of Quality of Service (QoS) constraints, depending on the level of overcommitment.

In this paper, we investigate a novel consolidation strategy based on an enhanced system model for the Infrastructure-as-a-Service cloud paradigm, which targets a better trade-off between Energy Efficiency and Quality of Service. We explicitly target modular cloud applications, which design is split into multiple components deployed in Virtual Machine (VM)s or containers. Our consolidation strategy allows to “freeze” parts of the application which are not currently used, making them available when requested with minimal latency. This improves energy saving with respect to other approaches, especially when idle VMs are present for backup or redundancy purposes, without degrading the service level. We compare multiple heuristics available in Optaplanner to solve our consolidation problem, and investigate improvements with respect to a more traditional approach. Our evaluation includes both simulations and experimentation in a real test-bed. The comparison shows that the Late Acceptance algorithm on average finds better solutions than other alternatives and energy efficiency improves up to 40% with respect to more conventional strategies, with deterioration of QoS indexes below 1%.

1. Introduction

Advanced design of cooling and power distribution systems have allowed the Power Usage Effectiveness (PUE) [1] of Data Center (DC)s to approach 1.10, which is close to the optimal limit [2,3]. However, regardless of this important trend, there are still several issues concerning the correct usage of this indicator [4]. Indeed, PUE is computed as the ratio between the total energy to operate the DC (including servers, cooling, lightning, power distribution losses) and the energy drawn by Information Technology (IT) equipment. The main problem is that the efficiency of the latter is not taken into consideration. In case of low server utilization, a relevant amount of energy is wasted to run idle equipment, so the required power for the whole infrastructure is disproportionate to the actual workload.

1.1. The problem of workload consolidation

While targeting an ideal linear relationship between power consumption and resource usage [5], power-saving mechanisms as frequency/voltage scaling, Low Power Idle (LPI), and sleep states [6] can be effectively used to reduce the impact of idle periods. Putting the whole device to *sleep*¹ has been demonstrated to be the most effective solution [6], because this totally avoids the bulk energy consumption of idle states. However, this mechanism can only be used when no application is running on the device, and this is not a typical condition for cloud infrastructures.

Workload consolidation enables to maximize the effectiveness of sleep states, by clustering together the largest number of VMs on the smallest number of servers. Only servers running users' applications are kept active, while idle servers are put to sleep. The selection of active servers is based on capacity constraints and communication patterns, so that even unused network links and devices could be put to sleep.

* Corresponding author.

E-mail addresses: alessandro.carrega@cnit.it (A. Carrega), matteo.repetto@cnr.it (M. Repetto).

¹ *Sleep* is a common term to indicate the Advanced Configuration and Power Interface (ACPI) S3 state [7].

The most challenging issue for workload consolidation is to find the correct balance between over-provisioning and performance constraints, especially for highly dynamic workloads. As a matter of fact, the more VMs are hosted on the same server, the less number of total active servers are needed, but this would likely lead to poor performance and violations of service level agreements. On the other hand, more conservative strategies could pursue minimal or no overcommitment at all, but the number of active servers would be larger and energy saving might become negligible. Many consolidation algorithms have already been proposed to this aim, which minimize the energy consumption of servers, network equipment or both.

1.2. Motivations for improved consolidation models

Despite of the large number of analytic models for data centers and their workloads, as well as alternative consolidation strategies, we argue that some important aspects have been largely neglected so far. Most consolidation algorithms group VMs based on resource constraints and usage, but they are totally unaware of their role in the cloud application they are part of. Indeed, today cloud applications are often designed with modular patterns, and multiple VMs are deployed for different functions. It is quite common to allocate more VMs than what strictly needed, for backup, redundancy, and scalability purposes, especially for real-time services. This is necessary to avoid degradation of QoS that occurs when additional VMs are provisioned in real time, but such instances remain idle most of their lifetime. This is a typical situation that occurs for many elastic services used with periodic patterns and variable workloads, as video streaming or transcoding; in some cases, the application only runs a few days per month, such as in the case of billing services. A plain consolidation strategy is not enough in these scenarios, since such idle VMs will contribute to keep up and running more servers than necessary, driving away from a straight linear relationship between utilization and power consumption.

1.3. Problem statement and approach

The presence of variable workloads and critical services often impacts the effectiveness of consolidation, since many VMs may run idle most of the time waiting for possible failures or peaks of requests. To overcome this problem, we advocate the adoption of a labeling system for VMs, which is used to identify their role and sensitivity to QoS aspects, including: i) over-commitment, ii) live-migration, and iii) suspend-to-RAM. Such classification could be easily inferred by meta-data and annotations that programmers are increasingly embedding into their cloud applications to drive life-cycle management operations carried out by software orchestration tools [8].

The presence of such label allows to distinguish between VMs that are expected to be busy most of the time and idle VMs that are only used occasionally. The former should be consolidated on active servers, whereas the latter can be left dispersed on the remaining servers and put to sleep. A Network Connectivity Proxy can be used to emulate the presence of idle VMs [9]; it wakes the corresponding servers up in a few hundred ms when they are expected to carry out some tasks, hence with minimal impact on service continuity and availability and a better trade-off between Energy Efficiency (EE) and service level.

In this work, we design and evaluate a workload consolidation strategy that is aware of the aforementioned labeling mechanism. We propose a system model that considers both servers and switching/routing devices for a cloud Infrastructure-as-a-Service (IaaS) paradigm; it is used to estimate how power consumption of the entire infrastructure changes with different placement decisions. A consolidation algorithm leverages this model to find the best configuration (i.e., placement of VMs on servers and forwarding decisions in network devices) that minimizes power consumption and does not violate QoS constraints.

Summing up, the main contributions of our paper are:

- a new system model that takes into account the labeling scheme;

- the introduction of multiple service classes for different usage patterns and QoS constraints;
- the comparison of multiple consolidation strategies to find out the most suitable one;
- the combination of both software simulations and real experimentation for numerical evaluation, which allows to consider practical aspects (packet loss, service downtime) hard to be realistically investigated only through simulations.

This paper extends our previous work [10], by: i) revising the definition of the system model and consolidation strategy; ii) extending simulations to resource usage under different optimization techniques; iii) including performance evaluation in a real testbed, through concrete implementation of the algorithms and their integration with OpenStack.

Comparison with a more traditional model shows that our approach improves energy saving when VMs do not run at full capacity all the time (which is a realistic condition in practice) and is closer to an ideal relationship between power consumption and workload. All the heuristics used to solve the consolidation problem improve energy saving up to 40% with respect to more conventional strategies; degradation of QoS indexes (namely packet loss and service interruption) is less than 1% with respect to the reference scenario. In addition, the ratio between power consumption and workload becomes linear starting from 50% system utilization, with an average improvement of around 30–40% over legacy approaches. Among the different heuristics, the Late Acceptance algorithm performs slightly better.

1.4. Structure of the paper

The paper is organized as follows. We briefly review related work about workload consolidation in Section 2. We define our model for the data center in Section 3 and the optimization problem in Section 4; the procedure for workload consolidation is then described in Section 5. We validate the proposed mechanisms by both simulation and experimentation in a real infrastructure in Section 6. Finally, we give our conclusion in Section 7, together with some indications for future work.

2. Related work

The consolidation problem has often been modelled as a modified bin-packing problem, seeking the optimal operating point between energy and performance [11]. Similar to our purpose, Shirayanagi et al. considered the presence of backup replicas and links, and investigated through simulations how power-consumption changes when varying their number [12]. However, the focus was on the power consumption of the network alone. Li et al. investigated the impact of overprovisioning (especially to deal with sudden peaks of workload) on power consumption [13]. Their strategy for workload consolidation considered only the servers and did not take into account any network metric.

Wang et al. [14] used a classical k -cut algorithm to distribute VMs in different racks as well as to select active network devices according to their communication patterns. Other authors have jointly modeled the power consumption of computing and networking devices in a more explicit way. The large number of discrete-values constraints for modeling the DC usually leads to NP-complete problems, so most authors fall back on heuristics and sub-optimal solutions [11,15,16].

Kliazovich et al. [17] considered both power consumption and performance, by relating server utilization with queue size. However, when both servers and network devices are involved, the size and complexity of the system quickly increase. Fang et al. [18] tackled this issue by solving the problem in three steps: i) they group VMs according to their traffic patterns; ii) they map such groups of VMs to racks in the data center; and iii) they define flow routing so to minimize the power consumption of the network infrastructure.

Despite the large number of algorithms proposed in the literature, the feasibility, effectiveness, and efficiency of workload consolidation

in real testbeds have not been fully investigated yet. Heller et al. analyzed how redundancy and guard margins influence the power consumption; their work places VMs with the objective of minimizing the power consumption of the data center network, but does not consider computing servers [19]. More recently, Beloglazov et al. proposed OpenStack Neat for balancing QoS and energy consumption [16]. Their framework monitors CPU utilization by VMs, estimates overloading/underloading conditions on each server, and uses OpenStack APIs for life-migration of VMs between servers. Oddly, their consolidation strategy only considers resource usage, but it is blind to power consumption.

There are many works aiming at the optimization of energy consumption without adversely affecting performance. For example, to mitigate the ecological impact, the major challenge of the work in [20] regards how to best select the most energy efficient services from cross-continental competing cloud-based data centers. This selection is addressed by Cloud-SEnergy, a system that uses a bin-packing technique to generate the most efficient service composition plans.

Fog-to-fog communication has been introduced to deliver services to clients with minimal reliance on the cloud through resource and capability sharing of cooperative fogs. Current solutions assume full cooperation among the fogs to deliver simple and composite services. The authors in [21] propose to dynamically create Service Level Agreement (SLAs) offered to cloud subscribers for service delivery while maximizing user satisfaction and fog profit gains. The solution provides a learning mechanism that relies on online and offline simulation results to build guaranteed workflows for new service requests. The configuration parameters of the short-term SLAs are obtained using a modified tabu-based search mechanism that uses previous solutions when selecting new optimal choices.

Cloud services are often distributed across several data centers requiring new scalable approaches to efficiently perform searching to reduce the energy and price cost of fulfilling requests. Multiagent-based systems have arisen as a powerful technique for improving distributed processing on a wide scale. They can operate in environments where partial observability is the norm and the cost of prolonged search can be exponential. In [22], the authors describe a multiagent-based service composition approach, using agent-matchmakers and agent-representatives, for the efficient retrieval of distributed services and propagation of information within the agent network to reduce the amount of brute-force search.

Finally, the continuing high rate of advances in information and communication systems technology creates many new commercial opportunities but also engenders a range of new technical challenges around maximising systems dependability, availability, adaptability, and auditability. The work in [23] focuses on a requirement model for the runtime execution and control of an intention-oriented Cloud-Based Application. Thus, a requirement modelling process referred to as Provision, Assurance and Auditing, and an associated framework are defined and developed where a given systems non-functional requirements are modelled in terms of intentions and encoded in a standard open mark-up language.

3. System model

The system model gives an abstract representation of the IaaS cloud paradigm suitable for analytic development. Our model describes the underlying infrastructure, virtual applications built on top of it, and their placement. The latter is given by allocation of VMs on servers and routing paths for network traffic. A power model describes the relationship between power consumption and resource usage, for each device in the infrastructure.

3.1. Data center infrastructure model

A typical data center is made of servers, clustered in racks, interconnected by a multi-layer hierarchy of network devices (either network

Table 1
Notation for the data center Model.

Parameter	Description
$\mathcal{G}(\mathbb{M}, \mathbb{E})$	Graph model of the DC infrastructure.
$\mathbb{M}$	Set of nodes (servers and network devices).
$\mathbb{S} / \mathbb{D}$	Sets of servers / network devices.
$(n, m) \in \mathbb{E}$	Directed edge from node $n \in \mathbb{M}$ to $m \in \mathbb{M}$.
c_n / r_n	Number of Central Processing Unit (CPU)s / amount of Random Access Memory (RAM) (in MB) available in server $n \in \mathbb{S}$.
b_{nm}	Bandwidth (Mbps) of network link between $n \in \mathbb{M}$ and $m \in \mathbb{M}$.
$\tilde{u}_{nm} \in [0, 1]$	Bandwidth usage on link $(n, m) \in \mathbb{E}$.
$\dot{u}_n \in [0, 1]$	Utilization of server/switch $n \in \mathbb{M}$.
$p_{nm} \in \mathbb{P}_{nm} \subseteq \mathbb{P}$	Network path from server $n \in \mathbb{S}$ to $m \in \mathbb{S}$.
$\mathbb{P}_{nm}$	Set of all network paths from server $n \in \mathbb{S}$ to $m \in \mathbb{S}$.
$\mathbb{P} = \bigcup_{n, m \in \mathbb{S}} \mathbb{P}_{nm}$	Set of all possible network paths.

switches or routers). As commonly done by other authors, we model the DC as a weighted directed graph $\mathcal{G}(\mathbb{M}, \mathbb{E})$. The set of nodes $\mathbb{M}$ includes both servers and network devices; they can be easily distinguished because servers are leaves in the graph, whereas network devices are intermediary nodes and roots. Servers are also indicated as the subset $\mathbb{S}$, network devices as the subset $\mathbb{D}$; clearly, $\mathbb{M} \equiv \mathbb{S} \cup \mathbb{D}$. Each network link is represented by a pair of edges in the graph, one for each direction; the generic link between $n, m \in \mathbb{M}$ is indicated as $(n, m) \in \mathbb{E}$.

The model also includes computing and networking resources associated to servers and links, respectively. For each server $n \in \mathbb{S}$, we consider both the number of CPUs c_n and the amount of RAM r_n . A CPU unit corresponds to the minimal hardware processing entity that could be put to sleep (e.g., core); memory capacity is measured in MB. We assume each network device $n \in \mathbb{D}$ can sustain the peak rate on all available interfaces. For each directional link $(n, m) \in \mathbb{E}$ we consider the network bandwidth b_{nm} , given in Mbps.

In a similar way, we also consider current utilization of resources, which is necessary to estimate power consumption and to avoid violating QoS constraints. Utilization is given as percentage with respect to available resources. $\dot{u}_n$ indicates the fraction of CPUs allocated to VMs on node $n \in \mathbb{M}$, and $\tilde{u}_{nm} \in [0, 1]$ indicates the bandwidth usage on link $(n, m) \in \mathbb{E}$. Both metrics are discussed in more details when defining the optimization constraints in Sub-Section 4.2.

The last notation in our model concerns end-to-end communication paths between VMs. Each path p_{nm} is represented by a finite and ordered sequence of directed links from source $n \in \mathbb{S}$ to destination $m \in \mathbb{S}$. Based on the network topology, multiple paths may be present between any couple of servers, for redundancy and/or load sharing. The set of all possible alternative paths between $n \in \mathbb{S}$ and $m \in \mathbb{S}$ is indicated as $\mathbb{P}_{nm}$; the set of all possible paths between any couple of servers is denoted as $\mathbb{P} = \bigcup_{n, m \in \mathbb{S} \wedge n \neq m} \mathbb{P}_{nm}$.

Table 1 summarizes all the notation described in this Section.

3.2. Virtualization model

The virtualization model describes cloud applications in terms of virtualization resources. We explicitly consider compound applications made of multiple software components, which are deployed in separate VMs. The set of all VMs instantiated in the DC at time $t \in [0, T]$ is indicated as $\mathbb{V}(t)$.

Our model can be seen as an extension of previous works [24,25]. Applications are modeled as arrival/departure processes that follow an exponential distribution. Differently from previous models, each arrival requests the allocation of a set $\mathbb{V}_k$ of VMs belonging to the same application at time t_v^a . The VM arrival and duration times are described by t_v^a and t_v^d , respectively. Another difference with the model in [25] is the usage of continuous time. Note that the model does not need to keep track of which application each VM belongs to, because any relation-

ship of this kind that is relevant for consolidation is already captured by communication traffic patterns.

From now on, we also use the notation $\mathbb{V}$ to refer to any of the two sets $\mathbb{V}(t)$ and $\mathbb{V}_k$ defined above.

The most remarkable novelty of our work is the modeling of the service level expected by each VM. While many VMs are always up and serving workloads, there will be a non-negligible number of them which are only provided for backup, resilience, and scaling purposes. They do not need to be up and running all the time, provided they can be made available with a very short latency (in the order of milliseconds). In addition, over-commitment can severely deteriorate the service level for VMs running close to their resource limit, but it is not expected to affect operation of low-loaded VMs. We therefore distinguish three distinct classes which imply different QoS requirements:

red [α_r]

for critical VMs (e.g., they are currently processing medium/high loads, hence require strict guarantees on resource availability (CPU, memory, and bandwidth);

yellow [α_y]

for VMs that are using less resources than what allocated (e.g., overcommitment may be used without affecting the service level); and

green [α_g]

for any other VM that is only provided for backup, resilience, scaling, and similar matters (e.g., they are not currently used for workload processing).

Each VM $v \in \mathbb{V}$ is therefore assigned to a specific class $\delta_v \in \mathbb{C}$. The set of these classes is indicated as $\mathbb{C} := \{\alpha_r, \alpha_y, \alpha_g\}$.

The three classes are mapped to different QoS constraints, especially for what concerns over-commitment. Each VM $v \in \mathbb{V}$ indicates its own resource requirements, including CPU $\tilde{c}_v$, RAM $\tilde{r}_v$, end-to-end bandwidth $\tilde{b}_{vw}$ with peer $w \in \mathbb{V}$. The degree of over-commitment is different for the three classes. In this work, we only consider the over-commitment ratio f_h (where $h \in \mathbb{C}$) for CPU, while additional constraints on memory, storage and bandwidth are left for future extensions.

Our model implicitly assumes that priority mechanisms are available in the underlying infrastructure to implement QoS according to this model. For example, the Linux *nice* feature [26] can be used to give precedence to the execution of red VMs, while multiple traffic queues [27] can be defined for scheduling of network packets.

3.3. Placement model

The placement model maps VMs and communication flows to physical servers and network links through the vectors $\mathbf{x} := [x_v | v \in \mathbb{V}]$ and matrix $\mathbf{y} := [y_{vw} | v, w \in \mathbb{V}]$, respectively.

Each element $x_v \in \{\mathbb{S} \cup \emptyset\}$ contains the identifier of the server that currently hosts the VM $v \in \mathbb{V}$. Similarly, each element $y_{vw} \in \{\mathbb{P}_{nm} \cup \emptyset\}$ contains the logical path from $v \in \mathbb{V}$ to $w \in \mathbb{V}$, given $x_v = n$ and $x_w = m$. Empty values ($\emptyset$) mean that the corresponding element is not currently instantiated; it may be a VM already terminated or rejected during the request due to insufficient resources.

A similar notation is used for internal variables of the consolidation procedure (see Section 5); they are used to store tentative configurations that are evaluated and modified during optimization iterations, before they are applied to the system at the end of the procedure. The vector $\hat{\mathbf{x}}$ and matrix $\hat{\mathbf{y}}$ denote the tentative placement of VMs and set of network paths, respectively. Each element $\hat{x}_v$ and $\hat{y}_{vw}$ has the same meaning of the corresponding variable x_v and y_{vw} .

We define the “distance” between the current system configuration $\{\mathbf{x}, \mathbf{y}\}$ and the tentative placement strategy $\{\hat{\mathbf{x}}, \hat{\mathbf{y}}\}$ in terms of the required VM migrations and changes in forwarding decisions. This parameter has a direct impact on service continuity. Migrations of VMs and changes of forwarding paths are indicated by the binary variables

Table 2
Virtualization model notation.

Parameter	Description
$t_v^a / t_v^d \in [0, T]$	Arrival / duration time (in s) of the VM $v \in \mathbb{V}$.
$\mathbb{C} := \{\alpha_r, \alpha_y, \alpha_g\}$	Set of VM classes with different QoS requirements.
$\delta_v \in \mathbb{C}$	Class to which a specific VM $v \in \mathbb{V}$ belongs.
$\tilde{c}_v / \tilde{r}_v \in \mathbb{R}_{\geq 0}$	Number of CPUs / amount of RAM (in MB) required by the VM $v \in \mathbb{V}$.
$\tilde{b}_{vw} \in \mathbb{R}_{\geq 0}$	Bandwidth (in Mbps) required between VMs $v \in \mathbb{V}$ to $w \in \mathbb{V}$.
f_h	Over-commitment ratio for CPU guaranteed to VMs belonging to the class $h \in \mathbb{C}$.

$\bar{x}_v$ and $\bar{y}_{vw}$, respectively, which are computed as follows (Table 2):

$$\bar{x}_v = \begin{cases} 1 & \text{if } x_v \neq \hat{x}_v \forall v \in \mathbb{V}(t) \\ 0 & \text{else} \end{cases} \quad (1)$$

$$\bar{y}_{v,w} = \begin{cases} 1 & \text{if } y_{vw} \neq \hat{y}_{vw} \forall v, w \in \mathbb{V}(t) \\ 0 & \text{else} \end{cases} \quad (2)$$

Accordingly, we compute two distances, as the number of migrations $\sum_{v \in \mathbb{V}} \bar{x}_v$ and changes of forwarding paths $\sum_{v,w \in \mathbb{V}} \bar{y}_{vw}$. In order to preserve the QoS settled by service level agreement, an upper bound can be set for both the number of migrations ($\bar{x}_*$) and re-routing of network paths ($\bar{y}_*$).

3.4. Power model

The power model captures the relationship between utilization and power consumption. It is needed to understand how power consumption changes with different placement decisions, so to drive the consolidation procedure. According to the main objective of our work, we define two power states which are commonly available in the hardware:

- sleep** (p_*) a very low-power state, where CPU is inactive (it is currently implemented by the ACPI S3 state²);
- active** (p_*) all the hardware is powered-on.

We assign the power state p_n according to current utilization, as a result of the consolidation process:

$$p_n = \begin{cases} p_* & \text{if } \dot{u}_n = 0 \\ p_* & \text{if } \dot{u}_n > 0 \end{cases} \quad (3)$$

Our power model reflects the typical non-linear behavior of IT devices [28,29]. The power curve Φ_n for every active device n is characterized by a bulk power consumption Φ_n^* in the idle state and an additional approximately linear dependence on the utilization $\dot{u}_n$ up to Φ_n^* at full load; the value is fixed at Φ_n^* in the sleep state (which is only possible in case of no utilization):

$$\Phi_n(\dot{u}_n) = \begin{cases} \Phi_n^* & \text{if } \dot{u}_n = 0 \\ \Phi_n^* + \dot{u}_n \cdot (\Phi_n^* - \Phi_n^0) & \text{if } \dot{u}_n > 0 \end{cases} \quad (4)$$

The definition of utilization $\dot{u}_n$ is based on resource consumption, and its calculation depends on the type of node.³ For servers, utilization is therefore the fraction of CPU assigned to VMs, taking into account the overcommitment ratio for each class:

$$\dot{u}_n = \frac{\sum_{\{v \in \mathbb{V} \mid x_v = n\}} \tilde{c}_v \times f_{\delta_v}}{c_n} \quad \forall n \in \mathbb{S} \quad (5)$$

For network devices, utilization is computed as the ratio between allocated bandwidth per egress and ingress path ($\tilde{b}_n$ and $\underline{\tilde{b}}_n$, respectively)

² Also known as suspend-to-RAM.

³ We do not consider utilization as the actual fraction of CPU usage, because this value is likely to have large fluctuations in the short term. Indeed, the time horizon of workload consolidation is usually longer, to avoid the overhead of migrations.

Table 3
Placement model notation.

Parameter	Description
$x_v \in \{\mathbb{S} \cup \emptyset\}$	Server where the VM $v \in \mathbb{V}$ is placed or a <i>null</i> value in case of a request rejection.
$\mathbf{x} := [x_v v \in \mathbb{V}]$	Vector of VM server assignments.
$y_{vw} \in \{\mathbb{P}_{nm} \cup \emptyset\}$	Network path for the communication from VM $v \in \mathbb{V}$ to $w \in \mathbb{V}$ (with $x_v = n$ and $x_w = m$) or a <i>null</i> value in case of a request rejection.
$\mathbf{y} := [y_{vw} v, w \in \mathbb{V}]$	Matrix of network path assignments.
$\dot{x}_v \in \mathbb{S}$	Current server hosting VM $v \in \mathbb{V}(t)$.
$\dot{\mathbf{x}} := [\dot{x}_v v \in \mathbb{V}(t)]$	Vector of servers hosting VMs.
$\dot{y}_{vw} \in \mathbb{P}_{nm}$	Current network path between VMs $v \in \mathbb{V}(t)$ and $w \in \mathbb{V}(t)$ (with $\dot{x}_v = n$ and $\dot{x}_w = m$).
$\dot{\mathbf{y}} := [\dot{y}_{vw} v, w \in \mathbb{V}(t)]$	Vector of network path assignments.
$\tilde{x}_v \in \{0, 1\}$	Indicates if the VM $v \in \mathbb{V}$ has to be migrated from a server to another one.
$\tilde{y}_{vw} \in \{0, 1\}$	Indicates if the selected network path for the communication from VM $v \in \mathbb{V}$ to $w \in \mathbb{V}$ is changed.
$\tilde{x}_* / \tilde{y}_*$	Upper bound on the number of migrations / network paths changes.

Table 4
Power model notation.

Parameter	Description
p_n	Power state of the device $n \in \mathbb{M}$.
p_*/p_*	<i>sleep</i> / <i>active</i> power state.
$\Phi_n(u_n)$	Power consumption of the device $n \in \mathbb{M}$.
$\Phi_n^* / \Phi_n^* / \Phi_n^*$	<i>sleep</i> / <i>idle</i> / <i>full</i> power consumption of the device $n \in \mathbb{M}$.
$\dot{u}_n$	Utilization level of the device $n \in \mathbb{M}$.
$\underline{b}_n / \underline{b}_n$	Allocated bandwidth for <i>egress</i> / <i>ingress</i> paths crossing the network device $n \in \mathbb{D}$.
$\underline{b}_n / \underline{b}_n$	Total capacity of outgoing / incoming links of the network device $n \in \mathbb{D}$.

and the total capacity of outgoing and incoming links ($\underline{b}_n$ and $\underline{b}_n$, respectively):

$$\dot{u}_n = \frac{\underline{b}_n + \underline{b}_n}{\underline{b}_n + \underline{b}_n} \quad \forall n \in \mathbb{D} \quad (6)$$

The expression for the total bandwidth allocated to network paths that cross a specific device is not trivial to describe. As a matter of fact, we need to consider all network links heading to the device; then, for each link, we must identify all paths that include it and sum up their allocated bandwidth. In addition, we should only consider the network paths for red and yellow VMs, since green VMs are idle and do not generate data. The following equations formalize this description for both the inbound and outbound direction (Table 3):

$$\underline{b}_n = \sum_{(n,m) \in \mathbb{E} \wedge m \in \mathbb{M}} \sum_{\substack{\{v,w \in \mathbb{V} \mid \\ (n,m) \in y_{vw} \wedge \\ \delta_v, \delta_w \neq \alpha_g\}}} \tilde{b}_{vw} \quad \forall n \in \mathbb{D} \quad (7)$$

$$\underline{b}_n = \sum_{(m,n) \in \mathbb{E} \wedge m \in \mathbb{M}} \sum_{\substack{\{v,w \in \mathbb{V} \mid \\ (m,n) \in y_{vw} \wedge \\ \delta_v, \delta_w \neq \alpha_g\}}} \tilde{b}_{vw} \quad \forall n \in \mathbb{D} \quad (8)$$

The total link capacity is simpler to describe, because we only have to sum up the capacity of every link heading at the device :

$$\underline{b}_n = \sum_{(n,m) \in \mathbb{E} \wedge m \in \mathbb{M}} b_{nm} \quad \forall n \in \mathbb{D} \quad (9)$$

$$\underline{b}_n = \sum_{(m,n) \in \mathbb{E} \wedge m \in \mathbb{M}} b_{nm} \quad \forall n \in \mathbb{D} \quad (10)$$

All the notation is also reported in Table 4.

4. Formulation of the consolidation problem

Workload consolidation aims at finding the best placement decision, which minimizes power consumption under domain, resource, and performance constraints. In this Section, we formulate the problem to be solved to compute optimal placement strategies.

4.1. Objective

The consolidation problem consists in finding the best placement of VMs and selection of network paths that minimize the total power consumption of the DC:

$$\min_{\mathbf{x}, \mathbf{y}} \sum_{n \in \mathbb{M}} \Phi_n(\dot{u}_n) \quad (11)$$

where $\Phi_n(\dot{u}_n)$ is given by the power model defined in Section 3.4.

The problem must be solved while taking into account some feasibility and performance constraints, which are described in Section 4.2. As a general indication, we aim at driving the consolidation strategy according to the following principles:

- cluster red and yellow VMs together on the smallest number of servers, according to capacity constraints and the overcommitment ratio for each class;
- leave green VMs on the remaining servers, without any explicit commitment to use the smallest number of devices;
- avoid service disruption and performance degradation by performing the minimal number of VM migrations and network reconfigurations.

4.2. Constraints

There are a number of constraints that must be considered when looking for a solution to (11). A first set of constraints concerns the feasibility of the solution, for instance: network paths originate and terminate on the servers where VMs are placed, links that build a network path must be consecutive, etc. The second set of constraints must consider the limits on available resources (CPU, memory, and bandwidth).

4.2.1. Domain constraints

The domains of allowed values for VM placement x_v and communication paths y_{vw} are limited to the set of servers $\mathbb{S}$ and network links $\mathbb{P}_{x_v x_w}$, respectively, plus the null value ($\emptyset$) in case no allocation is found:

$$x_v \in (\mathbb{S} \cup \emptyset) \quad \forall v \in \mathbb{V} \quad (12)$$

$$y_{vw} \in (\mathbb{P}_{x_v x_w} \cup \emptyset) \quad \forall v, w \in \mathbb{V} \quad (13)$$

4.2.2. Resource constraints

The allocation of VMs to servers is subject to availability of physical resources that fulfill the requirements on CPU and RAM. Overcommitment is only allowed for CPU, with a different ratio for each specific QoS class.

$$\sum_{\{v \in \mathbb{V} \mid x_v = n\}} \tilde{c}_v \times f_{\delta_v} \leq c_n \quad \forall n \in \mathbb{S} \quad (14)$$

$$\sum_{\{v \in \mathbb{V} \mid x_v = n\}} \tilde{r}_v \leq r_n \quad \forall n \in \mathbb{S} \quad (15)$$

In a similar way, the total bandwidth of paths crossing a given link must not exceed the link capacity:

$$\sum_{\{v, w \in \mathbb{V} \mid (n, m) \in y_{vw}\}} \tilde{b}_{vw} \leq b_{nm} \quad \forall (n, m) \in \mathbb{E} \quad (16)$$

4.2.3. Performance constraints

Frequent migrations and changes of network path negatively affect the service level [30]. Live migration of VMs takes time and consumes network bandwidth, due to the iterative memory-copy process [31]. Similarly, changes of switching rules may introduce forwarding loops, as well as delays or packet loss. In addition, there is a non-negligible risk for large fluctuations and oscillatory patterns in the allocation strategies, which become detrimental to the system operation. We therefore introduce a threshold on the maximum distance from the current placement to the new configuration, according to the definition that we have given in Section 3.3:

$$\sum_{v \in V} \bar{x}_v \leq \bar{x}_* \quad \sum_{v,w \in V} \bar{y}_{vw} \leq \bar{y}_* \quad (17)$$

While migration of VMs inescapably implies to change its related network paths, the latter could also change to achieve better utilization of the links, especially in case of arrival, departure, or migration of other VMs.

5. Consolidation strategy

The consolidation algorithm solves the problem described in Section 4. It initializes the system model described in Section 3 and uses it to estimate the overall power consumption for the different placement decisions. The problem is known to be NP-complete, so the solution is found by heuristic exploration of the space of possible states.

5.1. Consolidation procedure

A new consolidation procedure is started both at the reception of a new request $k \in \mathbb{N}$ and periodically. The last option is necessary to fix inefficiencies in the placement after termination of some VMs. Each request, in general, includes multiple VMs. The consolidation procedure tries first to find an optimal placement which include the new set of VMs; if not enough resources are available to accommodate every VM, those in the new request are selectively discarded until a solution is found⁴. The overall procedure is described by the pseudo-code in Algorithm 1.

Algorithm 1: Consolidation.

```

1 func CONSOLIDATION( $\mathbb{V}(t)$ ,  $\mathbb{V}_k$ ,  $\mathbf{x}$ ,  $\mathbf{y}$ ,  $\mathbb{M}$ ,  $\mathbb{E}$ ,  $\mathbb{C}$ ,  $\mathbb{P}$ )
2    $\mathbb{V}(t) \leftarrow \text{REMOVEDTERMINATEDVM}(\mathbb{V}(t))$ 
3   do
4     if OPTIMISE( $\mathbb{V}(t)$ ,  $\mathbb{V}_k$ ,  $\mathbf{x}$ ,  $\mathbf{y}$ ,  $\mathbb{M}$ ,  $\mathbb{E}$ ,  $\mathbb{C}$ ,  $\mathbb{P}$ ) =  $\emptyset$  then
5        $\mathbb{V}_k \leftarrow \text{CUTREQUEST}(\mathbb{V}_k, \mathbb{C})$ 
6     else
7       break
8     end if
9   while  $\mathbb{V}_k \neq \emptyset$ ;
10  return  $\mathbf{x}$ ,  $\mathbf{y}$ ,  $\mathbb{M}$ ,  $\mathbb{E}$ ,  $\mathbb{P}$ 
11 end

```

The first step is removing terminated VMs (line 2 of Algorithm 1). The criteria for this operation is straightforward: VMs are purged from the set $\mathbb{V}(t)$ when current time t is beyond their completion time, given by $t_v^a + t_v^d$ (see Algorithm 2). Indeed, this step is only required for simulations, whereas for the experimental setup VMs are automatically removed by the software orchestrator.

Once the set $\mathbb{V}(t)$ has been updated, the procedure computes a new allocation plan by solving the optimization problem described in Section 4 (line 4 of Algorithm 1). Several heuristics are compared for

⁴ We assume that a single VM may be discarded. The algorithm can be easily extended to cover the case of interdependency between VMs belonging to the same service, so to discard all linked VMs at the same step.

Algorithm 2: Remove terminated VMs.

```

1 func REMOVEDTERMINATEDVM( $\mathbb{V}(t)$ )
2   forall the  $v \in \mathbb{V}(t)$  do
3     if  $t_v^a + t_v^d \leq t$  then
4        $\mathbb{V}(t) \leftarrow \mathbb{V}(t) \setminus v$ 
5     end if
6   end forall
7   return  $\mathbb{V}(t)$ 
8 end

```

this purpose, as described in Section 5.2. If no solution is found, one VM is discarded according to the algorithm CUTREQUEST (line 5 of Algorithm 1), which pseudo-code is shown in Algorithm 3. This is nec-

Algorithm 3: CutRequest.

```

1 func CUTREQUEST( $\mathbb{V}_k$ )
2   forall the  $c \in \text{PRIORITYSORT}(\mathbb{C})$  do
3      $\tilde{\mathbb{V}}_k \leftarrow \{v \in \mathbb{V}_k \mid \delta_v = c\}$ 
4     if  $\tilde{\mathbb{V}}_k \neq \emptyset$  then
5        $\mathbb{V}_k \leftarrow \arg \max_{v \in \tilde{\mathbb{V}}_k} \leq \tilde{c}_v$ 
6       if  $|\tilde{\mathbb{V}}_k| > 1$  then
7          $\mathbb{V}_k \leftarrow \arg \max_{v \in \tilde{\mathbb{V}}_k} \leq \tilde{r}_v$ 
8       end if
9       return  $\mathbb{V}_k \setminus \text{RANDOM}(\tilde{\mathbb{V}}_k)$ 
10    end if
11  end forall
12  return  $\mathbb{V}_k$ 
13 end

```

essary, because over-commitment would otherwise lead to violation of QoS and SLAs.

The selection of the VM to discard is based on multiple criteria. A priority is assigned to each class, from the lowest value for green VMs to the highest value for red VMs (i.e., the PRIORITYSORT algorithm in line 2 of Algorithm 3). For the same value of priority, further criteria select VMs with the largest number of required CPUs first, and with the largest required amount of RAM then. In case more candidates are still present, one of them is chosen randomly.

The procedure described so far is then repeated until either a solution is found or all incoming VMs are discarded (lines 3–9 of Algorithm 1). In the last case, the original placement is returned, so there will be no changes.

5.2. Search methods and heuristics

Given the complexity of the problem, several works in the literature [15,18,32] propose to solve the problem into two sequential steps: i) optimal placement of VMs, without taking into account any communication constraint; and ii) optimal routing between VMs.

It is intuitive that splitting the problem may lead to sub-optima results; indeed, this approach does not allow either to find a closed-form solution nor to perform an exhaustive search (since the problem remains NP-hard). We therefore consider a single problem, and search for a joint placement and routing strategy that optimizes the cost function described in Section 5.

To this aim, we compare several heuristics available from the literature and we evaluate their effectiveness to find a solution in a reasonable period of time. To this purpose, we used the Optaplanner constraint satisfaction solver [33], which includes 3 families of optimization algorithms: i) Exhaustive Search (ES), ii) Construction Heuristics (CH), and iii) Meta-Heuristics (MH).

ES algorithms are the only ones that can find the global optimum, but their memory usage grows up exponentially so they can be used as reference only for small systems, which are not of interest in practice. The MH family is the recommended choice to deliver good results in a limited time. They should be used in combination with CH algorithms, which find a good initial solution in a finite length of time, yet they often lead to unfeasible solutions. Supported CH heuristics in Optaplanner are First Fit, First Fit Decreasing, Weakest Fit, Weakest Fit Decreasing, Strongest Fit, and Strongest Fit Decreasing.

We found that the alternative CH heuristics do not lead to very different results. Therefore, we generated the initial solution with Weakest Fit, which is generally quite good in all conditions. Then MH continued exploring additional states around this preliminary point, taking care of satisfying all constraints [34]. For this local search, we compared multiple options available in OptaPlanner [35]: Hill Climbing (HC) [36], Tabu Search (TS) [37,38], Simulated Annealing (SA) [39], and Late Acceptance (LA) [40].

6. Performance evaluation

The evaluation of the proposed model shows the trade-off between service level and EE. It is based on: i) simulations, for investigating the behavior while changing the topology and size of the system (Sub-Section 6.2); ii) an experimental test-bed, which takes into account the physical behavior of devices (Sub-Section 6.3).

The goodness of the solution found by the consolidation procedure is evaluated by comparison with a base scenario, denoted as Context and Power UnAware (CPUA) strategy. It acts like a sort of “blind” consolidation, by clustering VMs on the smallest number of servers without considering their power consumption and QoS class. The routing paths are computed after placement of VMs, hence behaving similar to other approaches already present in the literature. Apart neglecting power consumption, the CPUA strategy uses the same constraints we have defined in Section 5.

We investigate how different heuristics affect the efficiency of the consolidation algorithm, by considering complementary metrics. We take into account the following metrics:

- *energy-savings* with respect to CPUA consolidation;
- *acceptance-rate*, i.e. the average percentage of the accepted VM requests per minute;
- *red-yellow-under-satisfied*, i.e. the average percentage of allocated red and yellow VMs with less resources than required due to overcommitment;
- *yellow-resources*: i.e. the average CPU percentage and bandwidth available for the yellow VMs.

Energy saving is the explicit optimization target for the consolidation procedure. The set of constraints guarantees the correct level of over-commitment on processing and networking resources. However, heuristics do not fully explore the space of feasible solutions, so one method may provide better solutions than others. This also means that resource utilization may be sub-optimal, and some requests may be discarded unnecessarily. For this reason, we include the acceptance rate in the list of performance metric to compare the different heuristics.

6.1. Simulation settings

We consider a typical multi-rooted hierarchical topology for the Data Center Network (DCN), with three layers of switches (denote as core, aggregation, and edge), as shown in Fig. 1. A group of servers in a rack is connected to an edge switch, which is connected to aggregation switches. In turn, each aggregation switch is connected to a group of core switches.

We choose a fat-three topology [41,42], where the parameter v indicates the number of servers and network devices at each layer. This architecture also fixes the number of switch ports from the parameter v .

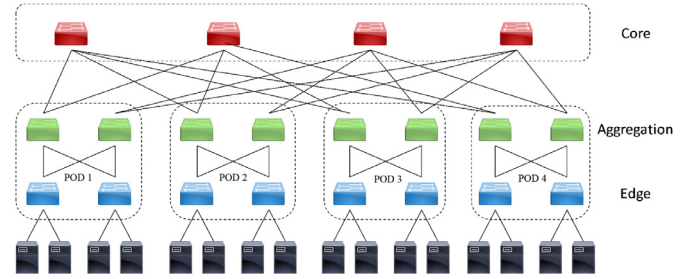


Fig. 1. Example of Fat-Tree DCN topology with $v = 4$ PODs.

Table 5

Power consumption values.

Device $n \in \mathbb{M}$	Φ_n^o	Φ_n^*	Φ_n^*
Server	2720 W	3520 W	≈ 5 W
Edge Switch	76.40 W	102 W	≈ 5 W
Aggregation Switch	133.50 W	175 W	≈ 5 W
Core Switch	555.00 W	656 W	≈ 5 W

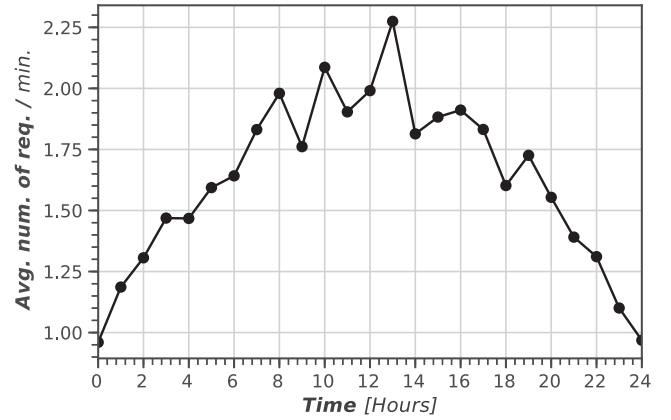


Fig. 2. Daily profile of the average number of requests per minute.

In the fat tree topology, there are v Performance Optimized Datacenter (POD)s that contain $v/2$ aggregation and $v/2$ edge switches. Each edge switch is connected to $v/2$ servers. At the highest layer, there are $v^2/4$ core switches, each one with a link to every POD. Therefore, a DCN with v PODs uses $5/4 v^2$ v -port switches and can support $v^3/4$ servers. An example of this topology (with $v = 4$ PODs) is shown in Fig. 1.

In our simulations, we choose $v = 8$ with a total number of switches and servers equal to 80 and 128, respectively, and set the link capacity to 1 Gbps. We assign the same amount of resources to every server: 128 GB of RAM and 32CPUs.

Table 5 shows the values of power consumption for servers and network devices. We chose the values according to references [19,29]. The sleep power, considering that most of the components are disabled and only the RAM is on, is usually less than 5 W for all devices.

We simulate the DC operation for $T = 24$ h, so to capture a typical daily workload. Unlike other works [25], the simulation time domain is continuous. Hence, a new request can arrive at any time between 0 and T hours.

Considering the results described in [43], we create a synthetic realization of the mean arrival rate of the incoming requests. Fig. 2 shows the average values of the typical daily profile indicated in [43]. The minimum value of 1request/min occurs overnight around 00:00. During the day, the mean request rate gradually grows up and overcomes 2requests/min around 12:00. In the afternoon, the average rate decreases, and it reaches the minimum value again during the night. Based on the

Table 6
Fraction of guaranteed CPU for class $h \in \mathbb{C}$.

Class	QoS class percentage	f_h
α_r	20%	1.0CPU
α_y	50%	0.2CPU
α_g	30%	0.0CPU

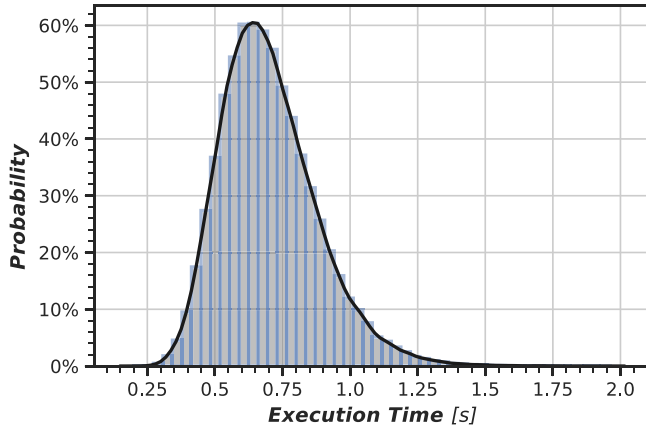


Fig. 3. Probability distributions of the execution time of the consolidation algorithm.

mean values shown in Fig. 2, we generate requests according to a truncated Log-normal distribution (tLN) during the day.

We also use the tLN for generating the number of VMs per request [25]. We vary the mean request duration in the range [1, 60] min using a tLN with mean value equal to 30 min. The number of CPUs required by a VM follows a tLN in the range [0, 10] CPUs with mean equal to $\bar{c} = 3CPU$ s. The RAM required by a VM follows a tLN in the range [1, 10] GB with mean $\bar{r} = 2$ GB. Finally, the bandwidth required by a VM is also generated by a tLN in the range [0, 200] Mbps, with mean $\bar{b} = 100$ Mbps.

Table 6 shows the percentage of assignments of VMs to QoS classes and the value f_h for each $h \in \mathbb{C}$.

Finally, the maximum distance between two consecutive allocation strategies is set proportionally to the number of VMs and network paths. Specifically, no more than 25% of VMs and 15% of network paths can be changed at each run of the consolidation algorithm.

6.2. Evaluation through simulations

We wrote an event simulator in Java using the Optaplanner constraint satisfaction solver [33] to evaluate the proposed algorithm. The validation takes into account different heuristics to solve the optimization problem (see Section 5.2), and their effectiveness to save energy and maximize the usage of the DC (i.e., to accept the largest number of VMs), with direct comparison with the CPUA. We repeated the experiments for a sufficient number of times to get small errors, until we reached the 95% Confidence Interval (CI)s. The measured CIs are not shown in the graphs since all of them are small and clutter the figures.

As performance indexes, we consider the time to solve the optimization problem and its effectiveness, namely energy saving with respect to the CPUA strategy. Fig. 3 shows the probability distribution of the execution time of the proposed algorithm. It mostly ranges in the interval [0.25, 1.25]s, therefore the consolidation procedure might be repeated very frequently. However, this is not feasible in practice, because migration of VMs and re-configuration of forwarding rules take much more time, as already demonstrated in [31].

Fig. 4 displays the average percentage of energy saving throughout the day, with respect to CPUA consolidation plan. The trend shows that great saving is achieved when the workload (i.e., the number of VMs)

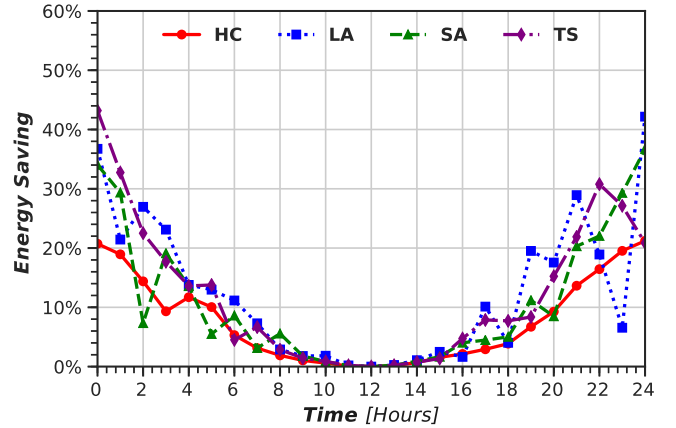


Fig. 4. Average percentage of energy saving with respect to the CPUA plan.

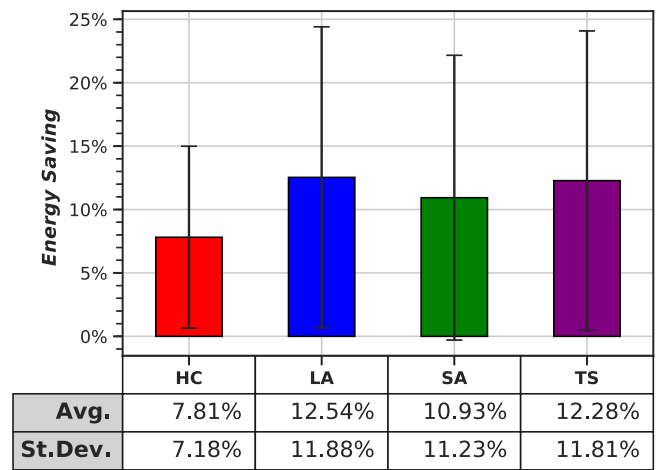


Fig. 5. Average percentage of energy saving and relative standard deviation with respect to the CPUA plan.

is low. This is expected, because with higher load there are less idle resources and less opportunity to apply power-saving mechanisms.

The results show a maximum energy saving of more than 40% with the LA heuristic. Compared to the other methods, HC obtains worse results because it easily gets stuck in local optima. Instead, the other heuristics implement more advanced solutions to avoid this problem. With a lower number of requests, energy saving is in the range [10, 30]%. The results have far larger variance for the TS, SA and LA heuristics than the HC. Indeed, the former methods exploit the concept of perturbations, which results in better exploration of the domain and more likelihood of jumping out of local minima; however, such perturbations also implies larger variability in the final result. The mean and standard deviation values for the considered heuristics are shown in Fig. 5.

Besides performance indexes, we also assess the service level in terms of accepted requests and allocated resources. The acceptance rate is shown in Fig. 6. On average, the LA method performs better than the other heuristics, which have similar results. Indeed, the advanced solutions introduced by LA [40] allow obtaining a better allocation of the resources with less power consumption. The CPUA strategy accepts less requests. The reason is that this strategy is unaware of QoS classes, so it guarantees the required resources to all VMs; this eventually results in a lower degree of overcommitment and more unused resources on average.

The knowledge of context information and QoS classes allows more aggressive overcommitment, therefore it is important to verify that such overcommitment does not impact the overall service level. We evaluate

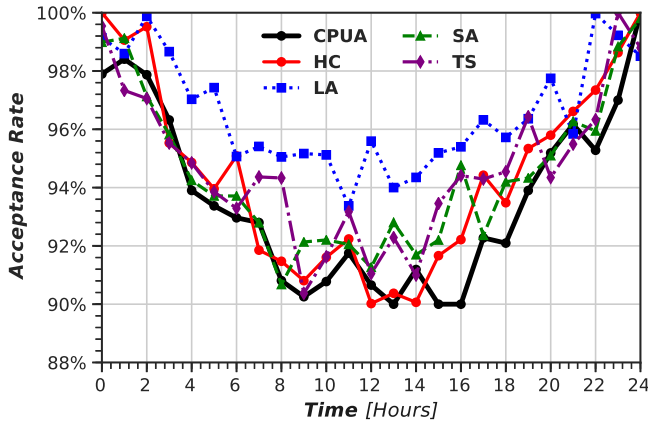


Fig. 6. Average percentage of accepted requests.

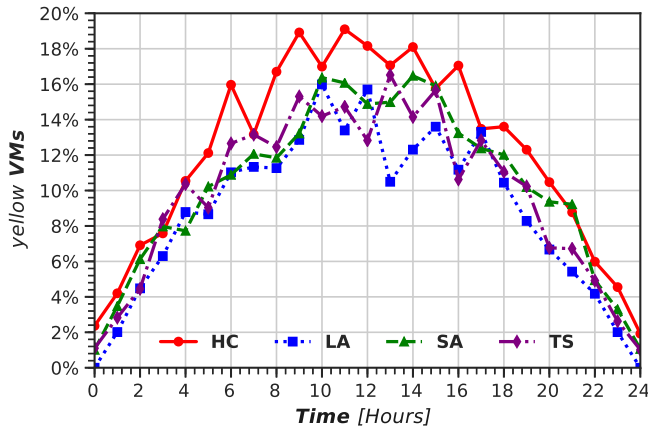


Fig. 7. Average percentage of yellow VMs with less assigned resources than required. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

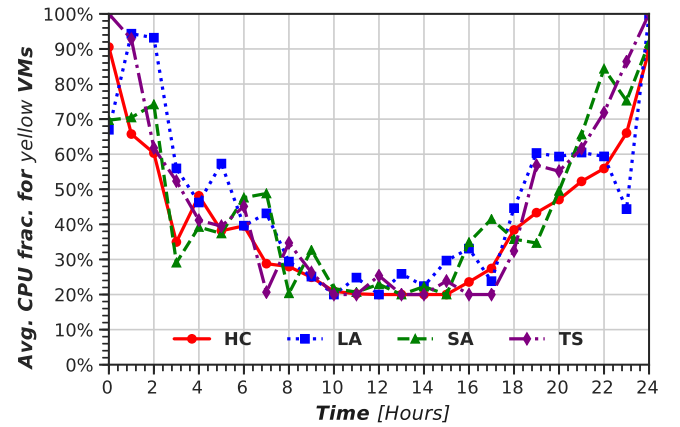
this aspect by taking into account the amount of resources available to yellow VMs. Indeed, the consolidation strategy only considers the guaranteed fraction f_{ay} of resources while grouping VMs; however, it is unlikely that the total amount of allocated resources perfectly saturates the server capacity, so there is some spare resources available to yellow VMs. Fig. 7 shows the average percentage of yellow VMs that are assigned less assigned resources than what requested.⁵ Similar to what observed for the acceptance rate, the LA algorithm provides better results. As a matter of fact, while allocating more resources to yellow VMs (Fig. 7) it also accepts the larger number of requests (Fig. 6), hence achieving better utilization of the system. With complementary considerations, we state that HC is the worst strategy, whereas the other options lie in between.

Finally, the statistics about average resources available for yellow VMs is shown in Fig. 8. These results confirm the previous findings: LA is the better heuristic, TS and SA lead to similar results, and HC is the worst choice in the set. We note, however, that differences in this case are lower than for previous metrics.

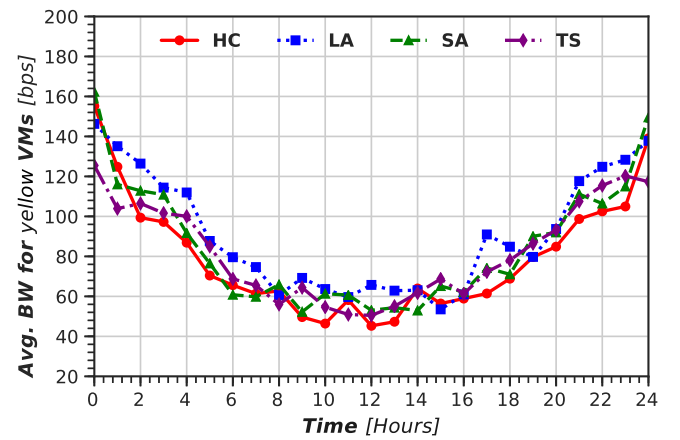
6.3. Evaluation in an experimental test-bed

In this Section, we validate the optimization algorithm in a real experimental test-bed with a sample application. The test-bed consists of a small OpenStack installation enhanced with additional control com-

⁵ We point out that in any case all accepted VMs are assigned enough resources as guaranteed by the overcommitment level of the specific class.



(a) CPU



(b) Bandwidth

Fig. 8. Average percentage of resources available for the yellow VMs. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

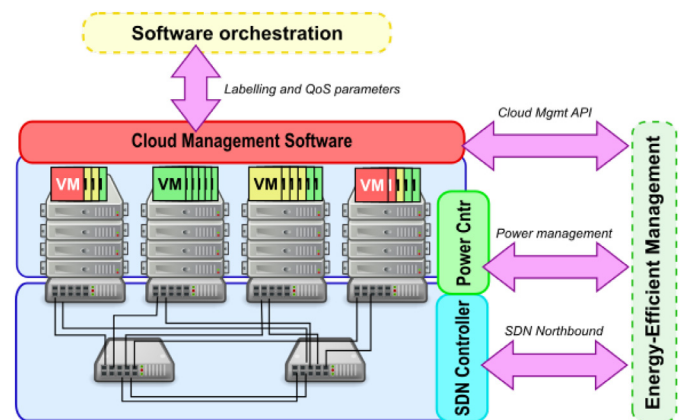


Fig. 9. Conceptual architecture for the energy-efficient test-bed.

ponents (as shown in Fig. 9), which implement all functionalities to manage power-management and QoS as required by the proposed consolidation mechanism [30]. The evaluation in this experimental test-bed allows the investigation and study of different technical aspects (e.g., non-ideal power consumption, packet loss and latency due to migration operations) while executing the consolidation procedure.

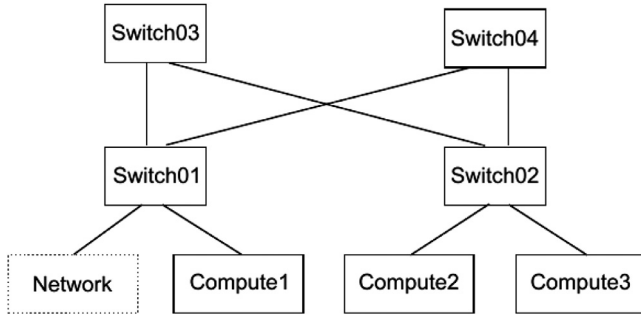


Fig. 10. Topology for the energy-efficient test-bed described in [30].

The layout of the test-bed we used in our experiments is depicted in Fig. 10. It does not include a large number of devices, but this is a realistic use case for edge computing.⁶ The size is comparable with similar experimental setups [13,16,45,46]. The infrastructure is composed of:

- OpenStack nodes: 3 compute nodes (Intel NUC6i7KYK, i7-6770HG 2.60GHz 4 cores, 4GB RAM), 1 network node, and 1 controller node (not shown in the picture);
- OpenFlow network: 4 OpenvSwitch instances running on a heterogeneous set of low-end commercial desktops (Atom, Pentium, Phenom; 2–4 cores; around 1.1GHz), heterogeneous links with 100Mbps/1Gbps capacity, and 1 control node running OpenDayLight (not shown);
- 1 storage node running Network File System (NFS) software (not shown); and
- management network on a separate infrastructure.

Additional details about the implementation and expected usage for energy-aware consolidation are given in our previous work [47].

For the evaluation, we developed a basic elastic transcoding application, made of one control master and several slaves. The master (dispatcher) loads video files and sends them to the slaves. The slaves are indicated as workers; they transcode the video files in parallel, one on each worker. This is representative of a large set of elastic applications, which could be implemented with big data frameworks (e.g., Hadoop, Spark, etc.). Since the main purpose is the evaluation of the proposed optimization algorithm, we do not consider more complex actions as video queuing or file splitting in multiple segments.

To simplify the evaluation of this experiment, we have 9 workers equally pre-allocated in the 3 compute nodes of the test-bed. Since orchestration is not the primary objective of our investigation, we used a naive policy, which wakes a worker up when a new request arrives and pauses it when it returns the result. The consolidation algorithm is run both periodically and when a new task arrives. To limit the computation burden, the algorithm is not executed when an existing request is completed. In this case, a sub-optimal condition will persist until the next periodic execution.

We generated request arrivals so to reproduce a larger and a smaller peak, as shown in Fig. 11. Initially, we generate 3 transcoding requests, one every minute, with video duration of 15 min, 19 min and 23 min, respectively. Then, after 3 min, we generated 6 additional requests (one every minute): 5 with the same video file of 22 min, and one with a bigger video of 27 min. Later on, 34 min after the begin of the simulation, we submitted 2 shorter videos (one every minute) of 6 min and 7 min, respectively.

Fig. 12 shows the workload during the experiment. At each time instant, the curve indicates the number of active transcoding tasks: there-

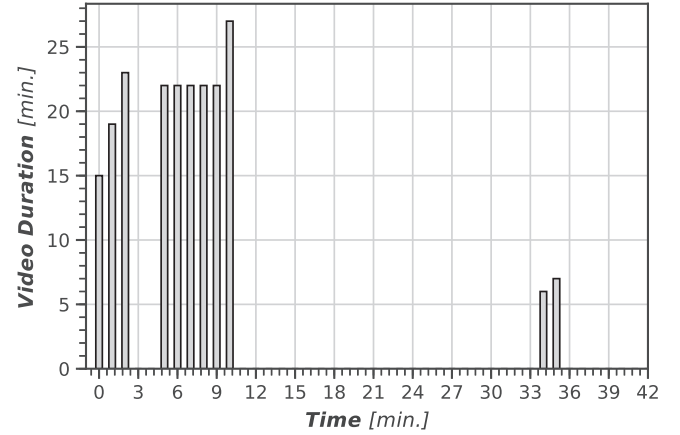


Fig. 11. Video arrivals and duration (in minutes) of the incoming transcoding requests during the experiment.

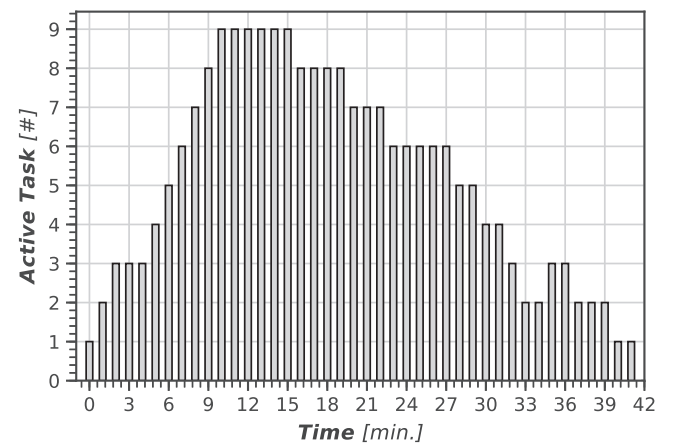


Fig. 12. Active transcoding tasks during the experiment.

fore each step up corresponds to the arrival of a new video and each step down to the completion of a transcoding request. As a consequence of the selected arrivals and duration times, the profile includes a 20 min traffic peak and a short 5 min spike. This situation is representative of sudden variations in workload, which are more difficult to tackle than steady state, and will be used to estimate the effectiveness of the consolidation algorithm under different parameters.

We repeated the same experiment until results lie in the 95% confidence interval. All the results described in this Section are the average of the repeated experiments. Similar to the previous simulations, we compare the results in two different scenarios: i) without context information and power-awareness (CPUA plan), and ii) using context information and power-awareness (with different heuristics: HC, LA, SA, and TS).

Fig. 13 shows and compares the energy saving with respect to CPUA consolidation plan got by the experiments. The utilization shape (corresponding to the workload profile) is reflected in power consumption, which increases when more tasks are present, hence leading to lower or null saving.

Without context information, VMs are treated uniformly by the consolidation procedure, independently of their actual utilization and role. In this case the number of allocated VMs basically prevents servers from sleeping; only one redundant switch can be put to sleep. When context information is used, there is the opportunity for larger energy saving, because there are idle VMs during the time interval under consideration. Clearly, saving energy is only possible when the system is partially used; we note that, power saving decreases with higher processing load.

⁶ Edge computing envisions the deployment of computing and storage resources at the network edge, which can be seen as small pervasive cloud installations. Edge installations are mainly conceived to provide low-latency services to local users, avoiding the round-trip delay to remote clouds [44].

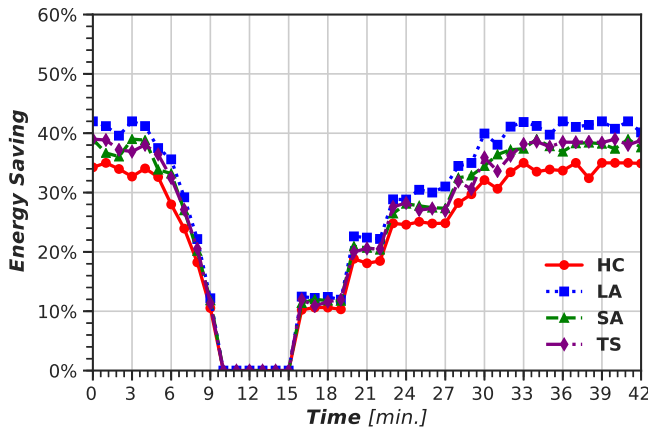


Fig. 13. Average energy saving with respect to CPUTA consolidation plan computed during the experiment.

Indeed, when the workload is low, energy saved amounts in the range [20, 40]%; the comparison between the different optimization heuristics confirms the general findings for simulations (i.e., LA yields to better results).

With respect to simulations, a real test-bed allows to better evaluate the impact of consolidation on QoS, which are affected by migrations of VMs between different servers. Therefore we investigated the impact of migrations on network traffic and service continuity.

Impact on network traffic is evaluated in terms of packet drops and jitters. Fig. 14 shows the probability distribution of the packet loss during the experiment. Generally, packet loss occurs during reallocation of VMs, due to the results of the consolidation strategy or when the system works at maximum and it is not able to satisfy the workload requirements. For this metric, HC and TS are the worst and best heuristics, respectively. Obviously, the CPUTA plan shows the best results because it reserves more resources for every VM during consolidation; however, every heuristic works well, being the packet loss always lower than 7%.

For service continuity, we consider the ratio between service interruption due to migration and total transcoding time for each video. Migrations suspend the execution of VMs, due to memory copy and re-routing of network streams [31]. This both impacts the interactivity of the service for real-time applications (which is not the case in our evaluation) and extends the overall execution. Low ratios affect the perceived quality of experience in a negligible way. Our experiments show that such ratio is usually below 1% for all heuristics but the HC algorithm, which however does not go beyond 1.7%. Compared to the CPUTA case, our consolidation strategy only degrades the service level by less than 0.5%. This is a good result, given the far larger improvement on energy efficiency (Fig. 15).

Finally, we investigated how consolidation affects the trade-off between performance and power consumption. Given the overall objective for workload consolidation, this is the most relevant parameter to estimate, though almost no other author provides its value. Fig. 16 shows the relationship between the workload (in terms of active transcoding tasks) and the power consumption in the different scenarios. The grey dashed line indicates an ideal linear relationship between performance and power consumption. Considering this ideal reference, the graph is divided into the following parts:

- red** when behaving worse;
- green** when behaving better.

The CPUTA scenario always performs worse than the proposed consolidation strategy. When the workload is below 50%, the behavior of our consolidation strategy is not very close to the optimal reference; however, it is rather better than the CPUTA model (nearly 50% lower). In this operational range, the small number of servers and tasks does not allow

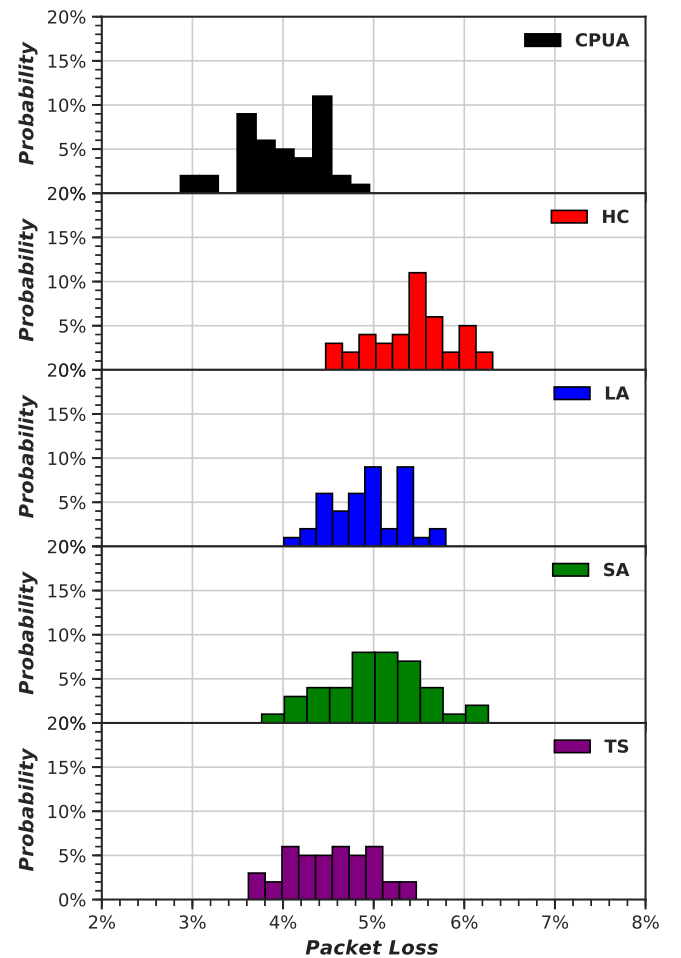


Fig. 14. Probability distribution of the packet loss during the experiment.

to completely saturate computing resources, so there are still many idle periods that hinder the achievement of better results. For higher workloads, it is more simple to saturate the resources of all servers, so in general better efficiency can be achieved. The proposed system model is able to effectively eliminate the impact of idle VMs, so the system already starts behaving very close to the optimal limit from 50% workload. In this operational range, the gap with the CPUTA model narrows, but does not become negligible. Once more, this graph confirms that LA is a better choice for our consolidation algorithm than the other heuristics.

7. Conclusions

In this paper, we have investigated energy efficiency for data centers. We have proposed an extended model for workload consolidation, which explicitly considers the presence of idle VMs, which are instantiated for resilience, backup, and availability purposes. As expected, our approach achieves better saving than legacy algorithms; in addition, we have demonstrated that the impact on QoS is almost negligible. Validation through simulations and real experimentation has allowed both the measurement of practical aspects (power consumption patterns of real devices, delays and overhead brought by migrations, packet losses) and the evaluation in larger scenarios. Comparison between heuristics has shown that the problem can be successfully solved by different algorithms; in general Late Acceptance performs slightly better than other alternatives.

Our work represents a further step towards a more linear relationship between power consumption and actual computation. Differently from

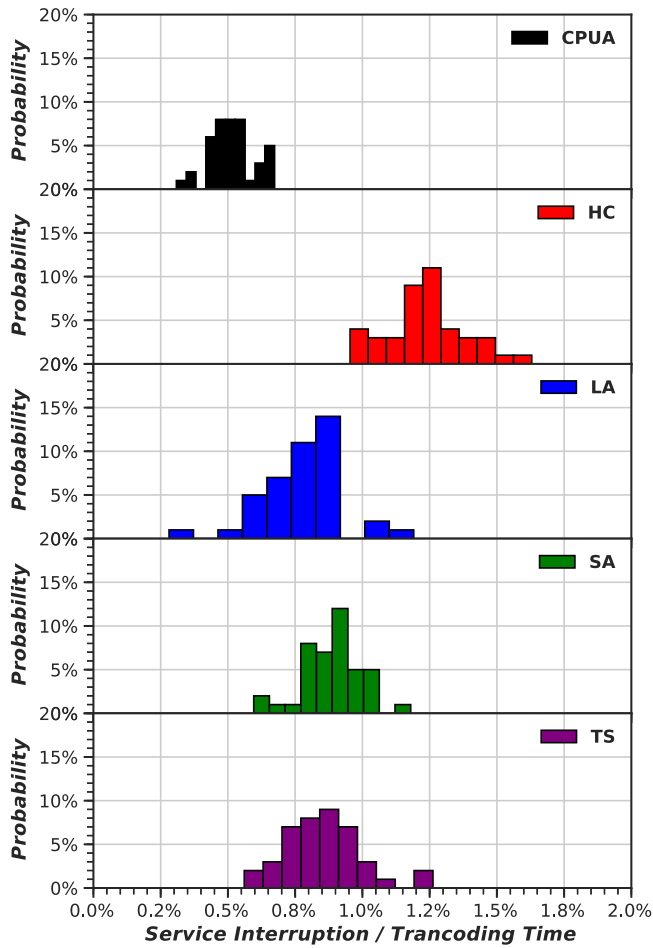


Fig. 15. Probability distribution of the service interruption / transcoding time ratio during the experiment.

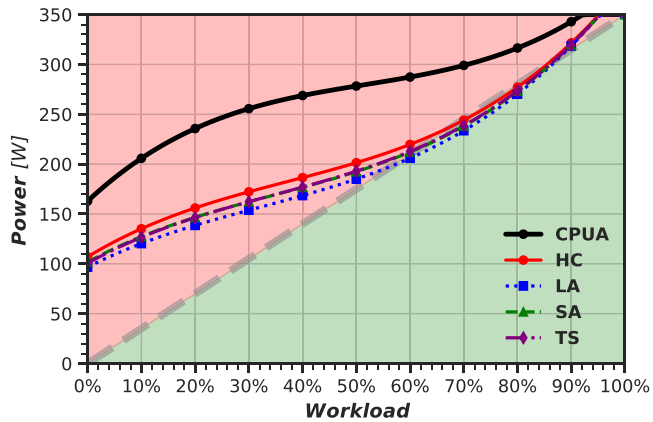


Fig. 16. Workload and power consumption relationship.

existing works, we have taken into account important factors in real data centers: idle VMs, different service level requirements, impact of migrations. This is clearly a meaningful effort that demonstrates how workload consolidation could be effectively applied to real systems.

We argue that the proposed mechanism relies on the willingness of users to declare QoS constraints for their workloads and to sleep their VMs. Future extensions should therefore consider incentive models that motivate users to take part in this mechanism, for example by pricing schemes that offer lower fares for “green” VMs. The optimization problem should be further extended in this direction, by translating power

consumption in operational costs and finding the optimal prices that maximize revenues for DC owners.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Alessandro Carrega: Software, Validation, Visualization, Investigation. **Matteo Repetto:** Data curation, Writing - original draft, Writing - review & editing.

Acknowledgement

This work was partially supported by the European Commission under the projects ARCADIA (contract 607881) and MATILDA (contract 761898).

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:[10.1016/j.comnet.2020.107210](https://doi.org/10.1016/j.comnet.2020.107210).

References

- [1] M. Rouse, What is power usage effectiveness (PUE)?
- [2] W. J. Kosik, Optimizing Power Usage Effectiveness In Data Centers, *www.esmagazine.com* (2007).
- [3] J. Yuventi, R. Mehdizadeh, A critical analysis of power usage effectiveness and its use in communicating data center energy consumption, *Energy Build.* (2013), doi:[10.1016/j.enbuild.2013.04.015](https://doi.org/10.1016/j.enbuild.2013.04.015).
- [4] G.A. Brady, N. Kapur, J.L. Summers, H.M. Thompson, A case study and critical assessment in calculating power usage effectiveness for a data centre, *Energy Convers. Manage.* (2013), doi:[10.1016/j.enconman.2013.07.035](https://doi.org/10.1016/j.enconman.2013.07.035).
- [5] L.A. Barroso, U. Hözl, The case for energy-proportional computing, *Computer* (2007), doi:[10.1109/MC.2007.443](https://doi.org/10.1109/MC.2007.443).
- [6] A. Hammadi, L. Mhamdi, A survey on architectures and energy efficiency in data center networks, *Comput. Commun.* 40 (2014) 1–21, doi:[10.1016/j.comcom.2013.11.005](https://doi.org/10.1016/j.comcom.2013.11.005).
- [7] Wikipedia, Advanced Configuration and Power Interface Specification - Rev 5.0 A, Wikipedia, 2010, doi:[10.1103/PhysRevB.67.245316](https://doi.org/10.1103/PhysRevB.67.245316).
- [8] T. OASIS, Topology and orchestration specification for cloud applications version 1.0, Organization for the Advancement of Structured Information Standards, 2013, doi:[10.1016/S0023-9690\(05\)80003-7](https://doi.org/10.1016/S0023-9690(05)80003-7).
- [9] R. Bolla, R. Khan, M. Repetto, Assessing the potential for saving energy by impersonating idle networked devices, *IEEE J. Sel. Areas Commun.* 34 (5) (2016) 1676–1689.
- [10] A. Carrega, M. Repetto, Cloud applications consolidation through context information and heuristic optimization, in: 28th International Telecommunication Networks and Applications Conference (ITNAC), 2018, pp. 1–7, doi:[10.1109/ATNAC.2018.8615406](https://doi.org/10.1109/ATNAC.2018.8615406).
- [11] S. Srikantiah, A. Kansal, F. Zhao, Energy aware consolidation for cloud computing, in: Proceedings of the 2008 Conference on Power aware Computing and Systems (HotPower), 2008, doi:[10.1016/j.jse.2014.12.017](https://doi.org/10.1016/j.jse.2014.12.017).
- [12] H. Shirayanagi, H. Yamada, K. Kono, Honeyguide: a VM migration-aware network topology for saving energy consumption in data center networks, *IEICE Transactions on Information and Systems*, 2013, doi:[10.1587/transinf.E96.D.2055](https://doi.org/10.1587/transinf.E96.D.2055).
- [13] B. Li, J. Li, J. Huai, T. Wo, Q. Li, L. Zhong, EnaCloud: an energy-saving application live placement approach for cloud computing environments, in: 2009 IEEE International Conference on Cloud Computing, 2009, pp. 17–24, doi:[10.1109/CLOUD.2009.72](https://doi.org/10.1109/CLOUD.2009.72).
- [14] L. Wang, F. Zhang, A.V. Vasilakos, C. Hou, Z. Liu, Joint virtual machine assignment and traffic engineering for green data center networks, *ACM SIGMETRICS Perform. Eval. Rev.* (2014), doi:[10.1145/2567529.2567560](https://doi.org/10.1145/2567529.2567560).
- [15] M. Mishra, A. Sahoo, On theory of VM placement: anomalies in existing methodologies and their mitigation using a novel vector based approach, in: Proceedings - 2011 IEEE 4th International Conference on Cloud Computing, CLOUD 2011, 2011, doi:[10.1109/CLOUD.2011.38](https://doi.org/10.1109/CLOUD.2011.38).
- [16] A. Beloglazov, R. Buyya, OpenStack neat: a framework for dynamic and energy-efficient consolidation of virtual machines in OpenStack clouds, *Concurr. Comput.* 27 (5) (2015) 1310–1333, doi:[10.1002/cpe.3314](https://doi.org/10.1002/cpe.3314).
- [17] D. Kliazovich, P. Bouvry, S.U. Khan, DENS: data center energy-efficient network-aware scheduling, *Cluster Comput.* (2013), doi:[10.1007/s10586-011-0177-4](https://doi.org/10.1007/s10586-011-0177-4).
- [18] W. Fang, X. Liang, S. Li, L. Chiaraviglio, N. Xiong, VMPlanner: optimizing virtual machine placement and traffic flow routing to reduce network power costs in cloud data centers, *Comput. Netw.* (2013), doi:[10.1016/j.comnet.2012.09.008](https://doi.org/10.1016/j.comnet.2012.09.008).

- [19] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, N. McKeown, ElasticTree: saving energy in data center networks, USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2010, doi:[10.1021/ci2004779](https://doi.org/10.1021/ci2004779).
- [20] T. Baker, B. Aldawsari, M. Asim, H. Tawfik, Z. Maamar, R. Buyya, Cloud-SEnergy: a bin-packing based multi-cloud service broker for energy efficient composition and execution of data-intensive applications, *Sustain. Comput.* 19 (2018) 242–252, doi:[10.1016/j.suscom.2018.05.011](https://doi.org/10.1016/j.suscom.2018.05.011).
- [21] I. Al Ridhawi, Y. Kotb, M. Aloqaily, Y. Jararweh, T. Baker, A profitable and energy-efficient cooperative fog solution for IoT services, *IEEE Trans. Ind. Inf.* 16 (5) (2020) 3578–3586, doi:[10.1109/TII.2019.2922699](https://doi.org/10.1109/TII.2019.2922699).
- [22] P. Kendrick, T. Baker, Z. Maamar, A. Hussain, R. Buyya, D. Al-Jumeily, An efficient multi-cloud service composition using adistributed multiagent-based, memory-driven approach, *IEEE Trans. Sustain. Comput.* (2018), doi:[10.1109/TSUSC.2018.2881416](https://doi.org/10.1109/TSUSC.2018.2881416).
- [23] T. Baker, M. Mackay, M. Randles, A. Taleb-Bendiab, Intention-oriented programming support for runtime adaptive autonomic cloud-based applications, *Comput. Electr. Eng.* 39 (7) (2013) 2400–2412, doi:[10.1016/j.compeleceng.2013.04.019](https://doi.org/10.1016/j.compeleceng.2013.04.019).
- [24] T. Benson, A. Anand, A. Akella, M. Zhang, Understanding data center traffic characteristics, *ACM SIGCOMM Comput. Commun. Rev.* (2010), doi:[10.1145/1672308.1672325](https://doi.org/10.1145/1672308.1672325).
- [25] A. Dalvandi, M. Gurusamy, K.C. Chua, Time-aware VMFlow placement, routing, and migration for power efficiency in data centers, *IEEE Trans. Netw. Serv. Manage.* (2015), doi:[10.1109/TNSM.2015.2443838](https://doi.org/10.1109/TNSM.2015.2443838).
- [26] nice. [Online]. Available: <https://www.linux.die.net/man/1/nice>
- [27] tc. [Online]. Available: <https://www.linux.die.net/man/8/tc>
- [28] P. Padala, K.G. Shin, X. Zhu, M. Uysal, Z. Wang, S. Singhal, A. Merchant, K. Salem, Adaptive control of virtualized resources in utility computing environments, *SIGOPS Oper. Syst. Rev.* (2007). 10.1.1.65.3456
- [29] P. Mahadevan, P. Sharma, S. Banerjee, P. Ranganathan, Energy aware network operations, in: *Proceedings - IEEE INFOCOM*, 2009, doi:[10.1109/INF-COMW.2009.5072138](https://doi.org/10.1109/INF-COMW.2009.5072138).
- [30] A. Carrega, G. Portomauro, M. Repetto, G. Robino, Boosting energy efficiency and quality of service through orchestration tools, *IEEE Cloud Comput.* 5 (6) (2018) 38–47, doi:[10.1109/MCC.2018.06418119](https://doi.org/10.1109/MCC.2018.06418119).
- [31] A. Carrega, G. Portomauro, M. Repetto, G. Robino, Energy efficiency for edge multimedia elastic applications, *Multimed. Tools Appl.* 78 (17) (2019) 24739–24764, doi:[10.1007/s11042-018-7050-x](https://doi.org/10.1007/s11042-018-7050-x).
- [32] A. Stage, T. Setzer, Network-aware migration control and scheduling of differentiated virtual machine workloads, in: *Proceedings of the 2009 ICSE Workshop on Software Engineering Challenges of Cloud Computing*, CLOUD 2009, 2009, doi:[10.1109/CLOUD.2009.5071527](https://doi.org/10.1109/CLOUD.2009.5071527).
- [33] G. De Smet et al., *OptaPlanner User Guide*, 2017.
- [34] Use Case: Machine Reassignment, 2019a. [Online]. Available: <https://www.docs.optaplanner.org/7.12.0.Final/optaplanner-docs/htmlsingle/#machineReassignment>.
- [35] Local Search, 2019b. [Online]. Available: <https://www.docs.optaplanner.org/7.23.0.Final/optaplanner-docs/htmlsingle/index.html#constructionHeuristics>
- [36] S. Russel, P. Norvig, *Artificial Intelligence A Modern Approach* 3rd Edition, Cambridge University Press, 2012, doi:[10.1017/S0269888900007724](https://doi.org/10.1017/S0269888900007724).
- [37] F. Glover, Tabu search part II, *ORSA J. Comput.* (1990), doi:[10.1287/ijoc.2.1.4](https://doi.org/10.1287/ijoc.2.1.4).
- [38] F. Glover, Tabu search part II, *ORSA J. Comput.* (1990), doi:[10.1287/ijoc.2.1.4](https://doi.org/10.1287/ijoc.2.1.4).
- [39] A. Khachatryan, S. Semenovskaya, B. Vainshtein, The thermodynamic approach to the structure analysis of crystals, *Acta Crystallogr. Sect. A* (1981), doi:[10.1107/S0567739481001630](https://doi.org/10.1107/S0567739481001630).
- [40] E.K. Burke, Y. Bykov, The late acceptance Hill-Climbing heuristic, *Eur. J. Oper. Res.* (2017), doi:[10.1016/j.ejor.2016.07.012](https://doi.org/10.1016/j.ejor.2016.07.012).
- [41] M. Al-Fares, A. Loukissas, A. Vahdat, A scalable, commodity data center network architecture, in: *Proceedings of the ACM SIGCOMM 2008 Conference on Data communication - SIGCOMM '08*, 2008, doi:[10.1145/1402958.1402967](https://doi.org/10.1145/1402958.1402967).
- [42] R. Niranjan Mysore, A. Pamboris, N. Farrington, N. Huang, P. Miri, S. Radhakrishnan, V. Subramanya, A.U.o.C. Vahdat, R.N. Mysore, Portland: a scalable fault-tolerant layer 2 data center network fabric, *SIGCOMM '09*, 2009, doi:[10.1145/1592568.1592575](https://doi.org/10.1145/1592568.1592575).
- [43] C. Oppenheimer, Which is less expensive: amazon or self-hosted?, 2012.
- [44] K. Bierzynski, A. Escobar, M. Eberl, Cloud, fog and edge: cooperation for the future? in: *Proceedings of the 2nd International Conference on Fog and Mobile Edge Computing (FMEC 2017)*, 2017. Valencia, Spain
- [45] W. Voorsluys, J. Broberg, S. Venugopal, R. Buyya, Cost of virtual machine live migration in clouds: a performance evaluation, in: M.G. Jaatun, G. Zhao, C. Rong (Eds.), *Cloud Computing*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 254–265, doi:[10.1007/978-3-642-10665-1_23](https://doi.org/10.1007/978-3-642-10665-1_23).
- [46] V. Cima, B. Grazioli, S. Murphy, T.M. Bohnert, Adding energy efficiency to Openstack, in: *2015 Sustainable Internet and ICT for Sustainability, SustainIT 2015*, 2015, pp. 1–8, doi:[10.1109/SustainIT.2015.7101358](https://doi.org/10.1109/SustainIT.2015.7101358).
- [47] A. Carrega, M. Repetto, G. Robino, G. Portomauro, OpenStack extensions for QoS and energy efficiency in edge computing, in: *2018 Third International Conference on Fog and Mobile Edge Computing (FMEC)*, 2018, pp. 50–57, doi:[10.1109/FMEC.2018.8364044](https://doi.org/10.1109/FMEC.2018.8364044).



Ph.D. Alessandro (aka Alex) Carrega is a Software and Network Engineer. Currently, he is a research associate at CNIT (National Inter-University Consortium for Telecommunications). He took part in the activities of many national and European projects (e.g. H2020 ASTRID, ARCADIA, GUARD and INPUT, FP7 IP ECONET, PRIN EFFICIENT, FIRB Green-Net, and FIWARE) and he is an active reviewer for many different international journals and conferences (IEEE and ACM). He has co-authored several papers in international conference proceedings. In 2010, he won the best paper award at the 3rd Int. Workshop on Green Communications (GreenCom 2010) co-located with the IEEE GLOBECOM Conference. In 2011 he was a Visiting Ph.D. Scholar at Portland State University (PSU), Portland, OR, USA under the supervisor of Prof. Suresh Singh as an active member of the FINE2 Italia USA collaboration project. He received the B.S. and M.S. degrees in Computer Engineering and the Ph.D. degree in Green Networking from the University of Genoa in 2005, 2007 and 2013, respectively. His research is on networking (energy-aware, performance optimization, and virtualization), software routers, NFV and SDN (in particular, OpenFlow). Finally, he is involved in collaboration with many industries, such as Telecom Italia, Broadcom, Nokia, Ericsson, Huawei, etc., and industrial fora, like GeSI.



Matteo Repetto is a research associate at CNIT. He has been teaching many courses in telecommunication networks and network security. He has been involved in many different national and European research projects in the networking area. His current research interests include pervasive and mobility management, energy-efficient networking, software-defined networking and network function virtualization, cloud computing.